Diagonalisation d'une matrice d'ordre 3

Durel TEUMO

2026-04-13

Contents

1 Création de la matrice A **2**

2 Détermination du polynôme caractéristique **2**
2.1 Création de la matrice identité d'ordre 3 2
2.2 Création d'une fonction pour le calcul de $p_A(\lambda) = det(A - \lambda \times I)$ 2

3 Détermination des valeurs et vecteurs propres de A **2**
3.1 Tracé de la courbe du polynôme caractéristique 2
3.2 Assignation des valeurs propres à des variables 3
3.3 Détermination des vecteurs propres 3

4 Construction de la matrice de passage P et de la matrice diagonale D **4**

5 Vérification de la diagonalisation **5**

1 Création de la matrice A

```
A <- matrix(c(1,0,-1, 1,2,1, 2,2,3), nrow=3, ncol=3, byrow = T)
A
```

```
##      [,1] [,2] [,3]
## [1,]    1    0   -1
## [2,]    1    2    1
## [3,]    2    2    3
```

On a une matrice carrée d'ordre 3.

2 Détermination du polynôme caractéristique

2.1 Création de la matrice identité d'ordre 3

```
I<- diag(3)
I
```

```
##      [,1] [,2] [,3]
## [1,]    1    0    0
## [2,]    0    1    0
## [3,]    0    0    1
```

2.2 Création d'une fonction pour le calcul de $p_A(\lambda) = det(A - \lambda \times I)$

Le polynôme caractéristique étant une fonction d'indéterminée λ, alors on peut utiliser `function()`, comme suit:

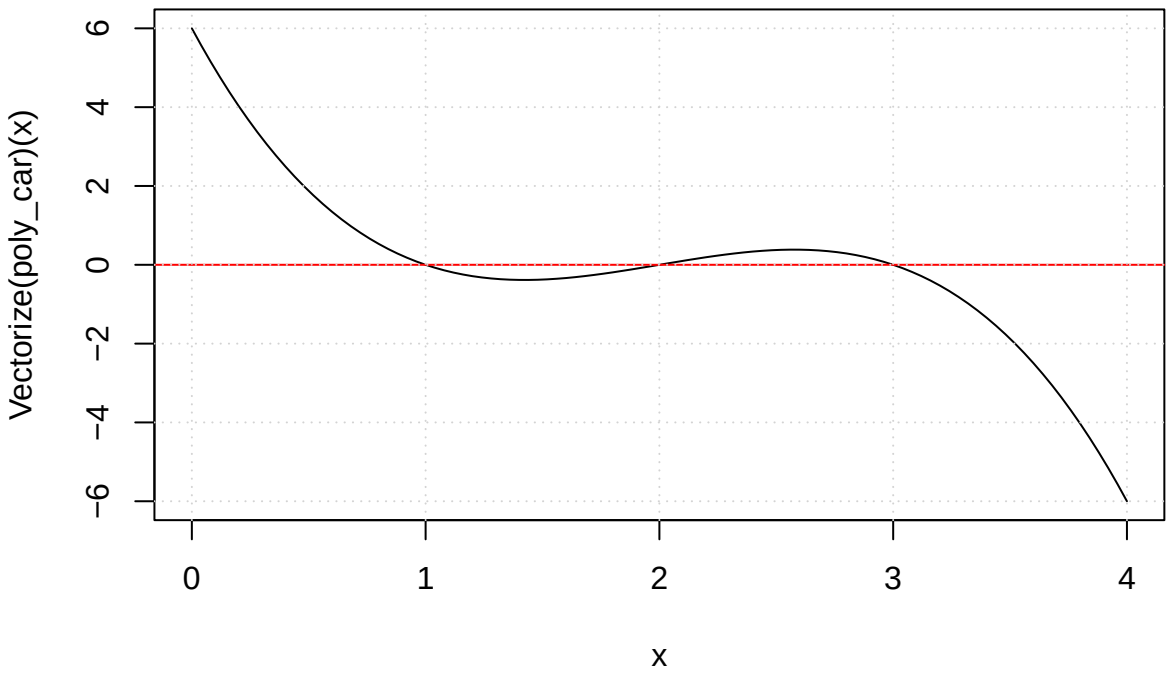
```
poly_car <- function(lambda) det(A - lambda * I)
```

3 Détermination des valeurs et vecteurs propres de A

3.1 Tracé de la courbe du polynôme caractéristique

Pour déterminer les valeurs propres de A sur $\mathbb{R}$, on peut d'abord faire une visualisation du polynôme avec **`curve ()`**, et ensuite utiliser **`uniroot`** pour attribuer chaque solution à une valeur précise. Une ligne rouge sera tracée avec **`abline()`** pour mettre en exergue l'axe des abscisses.

```
curve(Vectorize(poly_car)(x), from = 0, to = 4)
abline(h = 0, col = "red")
grid()
```



3.2 Assignation des valeurs propres à des variables

A partir de la question précédente, on voit clairement sur le graphique que les racines du polynôme caractéristique sont dans les intervalles [**0.5;1.5**], [**1.5;2.5**] et [**2.5;3.5**]. Les valeurs propres de la matrice A sont données par :

```
lambda_1 <- round(uniroot(Vectorize(poly_car), c(0.5,1.5))$root , 2)
lambda_2 <- round(uniroot(Vectorize(poly_car), c(1.5,2.5))$root , 2)
lambda_3 <- round(uniroot(Vectorize(poly_car), c(2.5,3.5))$root , 2)
c(lambda_1,lambda_2,lambda_3)
```

```
## [1] 1 2 3
```

Le polynôme caractéristique étant **scindé** et à **trois racines simples**, alors on peut conclure que **A est diagonalisable**.

Il nous reste donc à déterminer ses éléments de réduction.

3.3 Détermination des vecteurs propres

On peut utiliser la fonction **`solve()`** pour résoudre les équations matricielles d'indéterminée X de la forme $AX = b$:

```
# Pour lambda1
M <- A - lambda_1 * I

# On remplace la dernière ligne par (1, 0, 0) = 1
# pour fixer la première composante à 1
```

```r
M_mod <- M
M_mod[3, ] <- c(1, 0, 0)
b <- c(0, 0, 1)

v1 <- solve(M_mod, b)
v1
```

```
## [1]  1 -1  0
```

On peut vérifier le resultat:

```r
all.equal(A %*% v1, as.matrix(lambda_1 * v1))
```

```
## [1] TRUE
```

```r
# Pour trouver le vecteur propre associé à lambda_2
M <- A - lambda_2 * I

#Utilisons la Singular Value Decomposition de base de R
# La SVD décompose M et le dernier vecteur de V
# est toujours dans le noyau de M (solution de Mv=0)
v2 <- svd(M)$v[, 3]
v2
```

```
## [1]  0.6666667 -0.3333333 -0.6666667
```

On peut vérifier le resultat:

```r
all.equal(A %*% v2, as.matrix(lambda_2 * v2))
```

```
## [1] TRUE
```

```r
# Pour trouver le vecteur propre associé à lambda_2
M <- A - lambda_3 * I

#Utilisons la Singular Value Decomposition de base de R
# La SVD décompose M et le dernier vecteur de V
# est toujours dans le noyau de M (solution de Mv=0)
v3 <- svd(M)$v[, 3]
v3
```

```
## [1]  0.4082483 -0.4082483 -0.8164966
```

On peut vérifier le resultat:

```r
all.equal(A %*% v3, as.matrix(lambda_3 * v3))
```

```
## [1] TRUE
```

4 Construction de la matrice de passage P et de la matrice diagonale D

Des calculs précédents, on peut déterminer les éléments réducteurs de A:

```r
D <- diag(c(lambda_1,lambda_2,lambda_3))
D
```

```
##      [,1] [,2] [,3]
## [1,]    1    0    0
```

```
## [2,]    0    2    0
## [3,]    0    0    3
```

```
P  <- matrix(c(v1, v2, v3), nrow = 3, ncol = 3, byrow = F)
P
```

```
##      [,1]       [,2]       [,3]
## [1,]    1  0.6666667  0.4082483
## [2,]   -1 -0.3333333 -0.4082483
## [3,]    0 -0.6666667 -0.8164966
```

5 Vérification de la diagonalisation

Pour vérifier notre diagonalisation, il suffit de vérifier si :

$$A = PDP^{-1}$$

Calculons PDP^{-1}:

```
B<- P%*%D%*%solve(P)
round (B,2)
```

```
##      [,1] [,2] [,3]
## [1,]    1    0   -1
## [2,]    1    2    1
## [3,]    2    2    3
```

```
all.equal(B, A)
```

```
## [1] TRUE
```

La valeur retournée par le `all.equal(B,A)` étant **TRUE**, alors, on peut conclure que $A = PDP^{-1}$, donc A est bien diagonalisable.